HOSPITAL MANAGEMENT SYSTEM

Project Roadmap

A first-principles guide from models to deployed API

This document is your single source of truth for the project. Each phase builds on the last. Every task has a clear definition of done so you always know when to move forward — not just copy-paste and hope for the best.

How to Use This Document

This roadmap is structured around a simple principle: you only move to the next task when you can answer the "Definition of Done" question out loud — without looking at your code. If you can't explain it, you haven't learned it yet, you've only typed it.

The learning loop for each task

1. Read the task and its concept note before opening your editor.
2. Write the code yourself. Use this doc and Django docs, not AI autocomplete.
3. Test it manually via curl or a browser before marking it done.
4. Answer the Definition of Done question out loud (or write it down).
5. Only then move to the next task.

The tags in the task tables tell you what kind of work each task is:

MODEL	Defining database structure — Django models and migrations.

SERIAL	Serializers — converting model instances to/from JSON.
VIEW	Views — the logic that handles an HTTP request.
URL	Wiring a URL pattern to a view.
TEST	Verifying your code works correctly.
LOGIC	Business logic — methods, utilities, generators.
ADMIN	Registering models in Django admin for easy data inspection.

1 Foundation — Auth & User Profiles

Register · Login · Token · Who am I?

Why start here?

Every other feature in this project depends on knowing who is making the request. Before you can book an appointment, you need a patient. Before you can approve one, you need a doctor. JWT auth is the skeleton — everything else is muscle.

Key concept: AbstractBaseUser lets you replace Django's default username login with email. The role field on the User model is what your permission classes will interrogate on every request.

1.1 Patient Registration

A patient visits POST /api/auth/register/ with their details. Your view creates a User with role=patient, then creates a PatientProfile linked to it — both in the same request. The response returns JWT tokens so the user is immediately logged in.

#	Task	Type
1	**Write PatientRegisterSerializer** Fields: email, password, first_name, last_name, sex, age, weight, blood_group, allergies. validate() checks email uniqueness. create() uses a transaction to create User + PatientProfile atomically.	SERIAL
2	**Write PatientRegisterView** APIView with permission_classes = [AllowAny]. On POST: run serializer, call save(), generate JWT pair with RefreshToken.for_user(), return tokens + user data.	VIEW
3	**Add URL** POST /api/auth/register/ → PatientRegisterView	URL
4	**Test registration** curl POST with all fields. Check DB — both User and PatientProfile rows created. Try registering twice with same email — expect 400.	TEST

✦ **CHECK** Definition of Done: Can you explain why you need a transaction.atomic() when creating User + PatientProfile? What happens without it if the PatientProfile creation fails?

1.2 Login & Token Refresh

SimpleJWT provides the login view out of the box. Your job is wiring it and understanding what it returns.

#	Task	Type
5	**Wire login URL** POST /api/auth/login/ → TokenObtainPairView from simplejwt. This is already done if you followed the setup — verify it works.	URL
6	**Customise token claims** Subclass TokenObtainPairSerializer, override get_token() to add user's role and full_name into the JWT payload. Wire your custom serializer to the view.	SERIAL
7	**Test login flow** Login → copy access token → paste into Authorization: Bearer header → hit a protected endpoint. Then let token expire (lower lifetime in dev) and use refresh endpoint.	TEST

✦ **CHECK** Definition of Done: What is the difference between an access token and a refresh token? Why do we keep the access token short-lived?

1.3 Profile Endpoints

After logging in, a user should be able to see and update their own profile. A patient sees their PatientProfile. A doctor sees their DoctorProfile.

#	Task	Type
8	**PatientProfileSerializer** Read + write. For updates: partial=True so not all fields are required. Nest basic user fields (first_name, last_name) using a nested UserSerializer.	SERIAL
9	**DoctorProfileSerializer** Read-only for patients, read-write for the doctor themselves. Include department name via a nested DepartmentSerializer (read-only).	SERIAL

10	**GET + PATCH /api/auth/me/** RetrieveUpdateAPIView. In get_object(), return the profile based on request.user.role — patient gets PatientProfile, doctor gets DoctorProfile. No pk in URL.	VIEW
11	**Test profile update** PATCH with partial data (just weight, for example). Confirm only that field changes. Try hitting /me/ without a token — expect 401.	TEST

✦ **CHECK** Definition of Done: What does partial=True do in a serializer? Why is it important for PATCH vs PUT?

2 Departments & Doctor Browsing

List · Filter · Doctor detail

Why this phase next?

Before a patient can book anything, they need to browse. This phase is about read-only, public-facing data. It's a great phase to understand filtering, nested serializers, and read-only vs read-write permission splits.

2.1 Departments

#	Task	Type
12	**DepartmentSerializer** Simple ModelSerializer — id, name, description. No nesting needed.	SERIAL
13	**GET /api/departments/** ListAPIView. permission_classes = [AllowAny] — patients don't need to be logged in to browse departments.	VIEW
14	**Register Department in admin** So you can create test departments via /admin/ without needing an API endpoint for it yet.	ADMIN

✦ **CHECK** Definition of Done: Why is AllowAny appropriate here but not on the appointment endpoint?

2.2 Doctor Listing & Filtering

Patients browse doctors and filter by department. Each doctor card shows their name, department, specialty, and slot_duration.

#	Task	Type
15	**DoctorListSerializer** Flatten the nested User and DoctorProfile into one readable response. Include: full_name, email, department_name, specialty, degree, slot_duration.	SERIAL
16	**GET /api/doctors/** ListAPIView on DoctorProfile. permission_classes = [IsAuthenticated]. Override get_queryset() to filter by ?department= query param if provided.	VIEW
17	**GET /api/doctors/<id>/** RetrieveAPIView for a single doctor's full detail.	VIEW
18	**Test filtering** Create 2 doctors in different departments via admin. Hit /api/doctors/?department=1 — confirm only that department's doctors appear.	TEST

✦ **CHECK** Definition of Done: How does get_queryset() differ from queryset = ... on the view class? When would you use each?

3 Slot System

Availability · Leave · Generation · Manual slots

This is the most logic-heavy phase

The slot system is what makes this project non-trivial. You're not just doing CRUD — you're generating data from a template (availability), skipping exceptions (leave), and surfacing the result to patients as a clean list of bookable times.

Key concept: generators.py is pure Python logic, not a view. You test it in isolation. A view just calls it — thin views, fat logic.

3.1 Doctor Sets Availability

#	Task	Type
19	**DoctorAvailabilitySerializer** Write + validate. Check start_time < end_time. Check that changing availability doesn't conflict with already-booked slots (warn but don't block in v1).	SERIAL
20	**CRUD /api/slots/availability/** ListCreateAPIView + RetrieveUpdateDestroyAPIView. permission_classes = [IsDoctor]. In perform_create(), set doctor = request.user.doctor_profile automatically.	VIEW
21	**Test availability CRUD** Create Mon 09:00-13:00 for a doctor. Try creating another Mon entry — expect 400 (unique_together). Try as a patient — expect 403.	TEST

✦ **CHECK** Definition of Done: What does perform_create() do and why is it better than overriding create() in the serializer for setting the doctor field?

3.2 Doctor Leave

#	Task	Type
22	**DoctorLeaveSerializer** Validate that the leave date is in the future. Set doctor automatically on create.	SERIAL
23	**CRUD /api/slots/leave/** Same pattern as availability. IsDoctor permission. Doctor can only see/edit their own leave entries.	VIEW

3.3 Slot Generation

This is the core generator you already have in generators.py. Now you expose it and wire it to the system.

#	Task	Type
24	**Manually test generate_slots_for_doctor()** Open Django shell: from apps.slots.generators import generate_slots_for_doctor. Run it for a doctor who has availability set. Query Slot.objects.all() and confirm the rows match expected start/end times.	TEST
25	**Verify leave skipping** Add a DoctorLeave for a date that falls on their available day. Re-run generator. Confirm no slot was created for that date.	TEST
26	**Verify idempotency** Run the generator twice. Confirm slot count doesn't double. This works via ignore_conflicts=True in bulk_create.	TEST
27	**POST /api/slots/generate/** Thin view: IsDoctor permission, calls generate_slots_for_doctor(request.user.doctor_profile). Optional ?days_ahead= param. Returns count of slots created.	VIEW

✦ **CHECK** Definition of Done: Walk through the generator code line by line. What is bulk_create doing? What does ignore_conflicts=True mean and what problem does it solve?

3.4 Manual Slot Addition

#	Task	Type
28	**SlotSerializer (write)** Fields: date, start_time, end_time. Validate: date must be today or future. end_time must be after start_time. Set doctor and source=manual in perform_create().	SERIAL
29	**POST /api/slots/manual/** CreateAPIView. IsDoctor only. Doctor can add a one-off slot for any date/time.	VIEW

3.5 Patient Views Available Slots

#	Task	Type
30	**SlotSerializer (read)** Fields to expose: id, date, start_time, end_time, doctor_name, department. Never expose is_booked to patients — filter it out in queryset instead.	SERIAL
31	**GET /api/slots/?doctor=<id>** ListAPIView. IsAuthenticated. Filter: is_booked=False, is_active=True, date >= today, doctor=<id>. The ?doctor= param is required — return 400 if missing.	VIEW
32	**Test full slot flow** Set availability → generate → hit the list endpoint as a patient. Confirm only free, future slots appear. Mark one is_booked=True in admin. Confirm it disappears from list.	TEST

✦ **CHECK** Definition of Done: Why do we filter is_booked in the queryset rather than the serializer? What is the performance difference?

4 Appointments

Request · Approve · Reject · History

This is where all the pieces connect

The appointment system is where Patient, DoctorProfile, and Slot all come together. The key design challenge is keeping the slot state (is_booked) consistent with the appointment state (status). Your Appointment model already handles this in approve(), reject(), cancel() — your views just call those methods.

4.1 Patient Books an Appointment

#	Task	Type

33	**AppointmentCreateSerializer** Input: slot (id), reason_for_visit. Validate: slot exists, is_booked=False, is_active=True, slot.date is in the future. The patient is set from request.user in perform_create().	SERIAL
34	**POST /api/appointments/** CreateAPIView. IsPatient permission. On success: slot is held (is_booked=True), appointment status=pending. Return appointment id, slot details, status.	VIEW
35	**Test booking** Book a slot. Try booking the same slot again — expect 400. Try booking as a doctor — expect 403. Check DB — is_booked should now be True on the slot.	TEST

✦ **CHECK** Definition of Done: What is the race condition risk when two patients try to book the same slot simultaneously? How does the unique_together on Appointment + the is_booked check mitigate (but not fully eliminate) this?

4.2 Doctor Manages Appointments

#	Task	Type
36	**AppointmentDetailSerializer** Full read serializer. Include: patient full name, slot date/time, status, reason_for_visit, rejection_note, created_at.	SERIAL
37	**GET /api/appointments/pending/** ListAPIView. IsDoctor. Returns appointments where slot__doctor = request.user.doctor_profile and status=pending.	VIEW
38	**POST /api/appointments/<id>/approve/** Custom action view. IsDoctor + IsAppointmentDoctor (object permission). Call appointment.approve(). Return updated status.	VIEW
39	**POST /api/appointments/<id>/reject/** Same pattern as approve. Accept rejection_note in request body. Call appointment.reject(note=...). Confirm slot.is_booked flips back to False.	VIEW
40	**Test approve + reject** Book slot → approve as doctor → confirm status=approved and slot stays booked. Book another slot → reject → confirm status=rejected and slot is free again.	TEST

✦ **CHECK** Definition of Done: What is an object-level permission vs a view-level permission? Why do you need both IsDoctor and IsAppointmentDoctor on the approve endpoint?

4.3 History & Patient's Own Appointments

#	Task	Type
41	**GET /api/appointments/history/** IsDoctor. Returns appointments for their slots where status in [approved, completed, rejected]. Order by slot date descending.	VIEW
42	**GET /api/appointments/mine/** IsPatient. Returns all appointments for the logged-in patient. Include slot + doctor info.	VIEW
43	**POST /api/appointments/<id>/cancel/** IsPatient + IsOwnerPatient. Only allow cancellation if status=pending or approved. Call appointment.cancel(). Slot freed.	VIEW

✦ **CHECK** Definition of Done: Why should a patient only cancel pending or approved appointments — not rejected or completed ones? Enforce this in the view, not just the model.

5 Admin Operations

Doctor creation · Department management

Admin in DRF context

In this system, 'admin' means a user with role=admin, not the Django /admin/ panel (though you'll use that too). Admin API endpoints are only accessible to that role. This is where you practice the IsAdmin permission class.

#	Task	Type

44	**DoctorCreateSerializer** Admin creates a User with role=doctor and a linked DoctorProfile in one call. Fields: email, password, first_name, last_name, department, specialty, degree, slot_duration.	SERIAL
45	**POST /api/admin/doctors/** CreateAPIView. IsAdmin permission. On success return doctor id + login email.	VIEW
46	**CRUD /api/admin/departments/** ListCreateAPIView + RetrieveUpdateDestroyAPIView. IsAdmin. Admins can add/rename/delete departments.	VIEW
47	**GET /api/admin/doctors/** IsAdmin. List all doctors with their department and appointment count (annotate queryset).	VIEW
48	**Test admin permission wall** Try hitting POST /api/admin/doctors/ as a patient token — expect 403. As a doctor token — expect 403. Only admin token should get 201.	TEST

✦ **CHECK** Definition of Done: How does annotate() work in a queryset? Write the query that returns each doctor with their total appointment count as a single DB query.

6 Hardening & Polish

Errors · Validation · Select_related · Edge cases

Why this phase matters for learning

This is where junior devs stop and seniors keep going. The code works in the happy path — now you make it not break in the sad path. This phase teaches you the habits that make code production-quality.

6.1 Query Optimization

Every view that touches related models needs select_related or prefetch_related. Without it, Django fires one DB query per row — this is the N+1 problem.

#	Task	Type
49	**Audit all list views for N+1** In each ListAPIView, open Django shell and run the query with connection.queries. If doctor listing fires 1 query per doctor to get department — that's N+1. Fix with select_related.	TEST
50	**Add select_related to doctor list** get_queryset: DoctorProfile.objects.select_related('user', 'department'). Now it's 1 query total.	VIEW
51	**Add select_related to appointment list** Appointment.objects.select_related('patient__user', 'slot__doctor__user', 'slot__doctor__department'). Confirm query count drops.	VIEW

✦ **CHECK** Definition of Done: Explain the difference between select_related and prefetch_related. Which one would you use for a ForeignKey and which for ManyToMany?

6.2 Error Handling & Validation

#	Task	Type
52	**Consistent error responses** Create a utils/exceptions.py with a custom exception handler. Register it in settings REST_FRAMEWORK EXCEPTION_HANDLER. All errors should return {error: ..., detail: ...} not Django's default format.	LOGIC
53	**Validate slot booking window** In AppointmentCreateSerializer: reject if slot.date is more than 30 days in the future (configurable). Add a clear error message.	SERIAL
54	**Doctor can't book appointments** This should already be handled by IsPatient — but write a test that explicitly tries it to confirm. Document why the permission is at view level and not serializer level.	TEST

6.3 Pagination

#	Task	Type
55	**Add PageNumberPagination** In settings, set DEFAULT_PAGINATION_CLASS and PAGE_SIZE=20. Test that /api/slots/ returns paginated results with next/previous links.	LOGIC
56	**Test with large data set** Use Django shell to bulk-create 50 slots. Hit the endpoint and confirm pagination is working. Confirm ?page=2 returns the next page.	TEST

7 Automated Tests

Write tests that prove your code works

Testing from first principles

You've been manually testing with curl this whole time. Now you write code that does the testing for you, so regressions get caught automatically. DRF provides APIClient — a test HTTP client that you use exactly like curl, but in Python.

Key concept: Each test should test ONE thing. If a test for approval also tests login, it's two tests masquerading as one. Write narrow, focused tests.

#	Task	Type
57	**Write test for patient registration** TestCase: valid registration creates User + PatientProfile. Invalid registration (duplicate email) returns 400. Test both happy and sad path.	TEST
58	**Write test for slot availability** TestCase: doctor can create availability. Patient cannot (403). Duplicate day returns 400.	TEST
59	**Write test for appointment lifecycle** Full integration test: create patient + doctor + slot → patient books → doctor approves → confirm slot is_booked=True. Then doctor rejects another → slot is_booked=False.	TEST
60	**Write test for generator** Unit test for generate_slots_for_doctor(). Set availability Mon 09:00-11:00, slot_duration=60. Run generator for a Monday date. Confirm exactly 2 slots created with correct times.	TEST

61	**Write test for permission walls** One parametrized test that hits every protected endpoint with wrong role tokens and confirms 403 across the board.	TEST

✦ **CHECK** Definition of Done: Run python manage.py test and all tests pass. No test is asserting something you didn't explicitly intend to test.

What Comes After

Once all 7 phases are done and tests pass, this project is genuinely complete for its scope. Here is what a real production extension would look like — don't build these now, but know they exist:

#	Task	Type
—	**Celery + Redis for scheduled slot generation** Replace the manual POST /generate/ with a nightly Celery beat task that regenerates slots for all doctors automatically.	LOGIC
—	**Email notifications** Send confirmation email on appointment approval/rejection using Django's send_mail or a service like SendGrid.	LOGIC
—	**OpenAPI / Swagger docs** Add drf-spectacular to auto-generate interactive API docs at /api/docs/. Useful for frontend devs consuming your API.	LOGIC
—	**Docker + PostgreSQL** Containerise the app. Switch from SQLite to PostgreSQL using the production settings you already have.	LOGIC

You started this project feeling like things were abstract. The antidote to abstraction is not reading more — it's building one concrete thing at a time and making sure you can explain it before moving on. Follow this doc phase by phase and you'll finish with both a working project and a genuine understanding of how DRF fits together.